

UD 06.3. JAVASCRIPT 3

LM

Ciclos Formativos Grado Superior

Pascual Ligeró
pa.ligero2@edu.gva.es
2025/2026
Versión:260321.2011

Licencia



Reconocimiento - NoComercial - Compartirlgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

⚠ Importante

⚗ Interesante

ÍNDICE DE CONTENIDO

1. Conceptos: Sincronía vs Asincronía.....	5
2. Promesas (El Futuro del Código).....	5
2.1 Estados de una Promesa.....	5
2.2 Ejemplos.....	6
2.3 Errores Típicos.....	6
3. Async / Await.....	6
3.1 Ejemplos.....	7
3.2 Manejo de Errores (Try / Catch).....	7
3.3 Errores Típicos.....	7
3.4 Mini-reto (5 min).....	8
4. Fetch API + HTTP + JSON.....	8
4.1 Métodos HTTP (Verbos).....	8
4.2 Ejemplos con Fetch.....	8
4.2.1 Petición GET Básica.....	8
4.2.2 Petición POST (Crear).....	9
4.3 Errores Típicos.....	10
4.4 Mini-reto (5 min).....	10
5. Cheat Sheet.....	10
6. Preguntas Rápidas (V/F).....	11
7. Checklist de Habilidades.....	11
8. Ejercicios Finales (La Práctica).....	11
8.1 Nivel 1: Calentamiento (Fácil).....	11
8.2 Nivel 2: Intermedio (Flujos Reales).....	12
8.3 Nivel 3: Boss Level (Mini-Cliente API).....	13
9. Almacenar información en el navegador.....	14
9.1 Localstorage y Sessionstorage.....	14
9.2 Ejemplo sencillo con localStorage.....	14
9.3 Ejemplo sencillo con sessionStorage.....	15
10. prácticas integradoras.....	15
10.1 El Buscador de Usuarios "En Vivo" (RandomUser API).....	16
10.2 La Pokedex (Promise).....	16
10.3 Dashboard Crypto con "Auto-Refresh".....	17
10.4 Fake E-commerce: Carrito Persistente.....	17
10.5 GitHub User Finder (Manejo de Errores 404).....	18
10.6 Blog con "Optimistic UI" (POST & DELETE).....	18
10.7 Conversor de Monedas (Math + Objetos).....	18
10.8 El "Smart Search" con Debounce (Nivel Boss).....	19
10.9 La Carrera de Servidores (Promise.any).....	19
10.10 Gestor de Tareas con Estados (Async + ClassList)	19
11. Bibliografía.....	21

UD06.3. JAVASCRIPT 3

1. CONCEPTOS: SINCRONÍA VS ASINCRONÍA

Sincronía (Bloqueante): Una fila en el supermercado con un solo cajero. Si el cliente actual tarda media hora buscando monedas, NADIE más pasa. El código se detiene.

Asincronía (No bloqueante): Pedir comida en un restaurante. Pides (lanzas la tarea), te sientas a hablar (haces otras cosas) y, cuando la comida está lista, te avisan (callback/promesa) y te la traen.

JavaScript

```
// 1. Código Síncrono (Orden lógico)
console.log("1. Inicio");
console.log("2. Proceso");
console.log("3. Fin");
// Salida: 1, 2, 3

// 2. Código Asíncrono (El Salto Temporal)
console.log("A. Inicio");

// setTimeout manda esto al "futuro" (mínimo 0ms, pero va a la cola)
setTimeout(() => {
  console.log("B. Dentro del timeout");
}, 0);

console.log("C. Fin");

// Salida REAL: A -> C -> B
// (Aunque sea 0ms, JS termina lo síncrono antes de mirar la cola)
```

2. PROMESAS (EL FUTURO DEL CÓDIGO)

Promesa: Un objeto que representa un valor que puede estar disponible ahora, en el futuro, o nunca. Es como un ticket de pedido de hamburguesa.

2.1 Estados de una Promesa

1. **Pending (Pendiente):** Estás esperando (la hamburguesa se está cocinando).
2. **Fulfilled (Resuelta):** ¡Éxito! (Te entregan la hamburguesa).
3. **Rejected (Rechazada):** Error. (Se quemó la cocina, no hay hamburguesa).

2.2 Ejemplos

JavaScript

```
// 1. Crear una Promesa (Simulacro)
// resolve, reject podrían llamarse como queramos, pero es un estandard
const chequearWifi = new Promise((resolve, reject) => {
    const hayInternet = true; // Prueba cambiando a false

    setTimeout(() => {
        if (hayInternet) {
            resolve("Conectado a la red"); // Éxito
        } else {
            reject("No hay señal"); // Fallo
        }
    }, 5000); // pasados 5 segundos se resuelve
});

// 2. Consumir la Promesa (.then / .catch)
console.log("Buscando red...");

chequearWifi
    // Se ejecuta si resolve() mensaje es la cadena que devuelve resolve
    .then((mensaje) => {
        console.log(mensaje); // Se ejecuta si resolve()
        return "Abriendo Google..."; //Podemos encadenar pasos con otros .then y reciben el mensaje
    })
    .then((otroMensaje) => {
        console.log(otroMensaje); // Recibe el return del anterior
    })
    // Se ejecuta si reject() error es la cadena que devuelve reject
    .catch((error) => {
        console.error(error);
    })
    .finally(() => {
        console.log("Proceso terminado (pase lo que pase)");
    });
```

2.3 Errores Típicos

- No poner `.catch()`. **Corrección:** Si una promesa falla y no hay catch, verás un error feo de "Uncaught (in promise)" en la consola.

3. ASYNC / AWAIT

Async/Await: Es una forma más limpia de escribir promesas. Hace que el código asíncrono *parezca* síncrono (se lee de arriba abajo).

- **async:** Convierte una función en una que devuelve siempre una promesa.
- **await:** "Pausa" la ejecución de la función *en esa línea* hasta que la promesa se resuelva. **Importante:** Solo pausa esa función, no bloquea el navegador.

3.1 Ejemplos

JavaScript

```
// Simulamos una petición
const esperar = (ms) => new Promise(resolve => setTimeout(resolve, ms));

// Versión como antes (.then)
/*
function iniciar() {
    esperar(1000).then(() => console.log("Listo"));
}
*/

// Versión Moderna (Async/Await)

const iniciarSistema = async () => {
    console.log("Iniciando...");

    // El código se detiene AQUÍ 1 segundo (pero la web sigue viva)
    await esperar(1000);

    console.log("Sistema cargado ");
};

//main
iniciarSistema();
console.log("Esto sale ANTES que 'Sistema cargado'");
```

3.2 Manejo de Errores (Try / Catch)

Con async/await, volvemos a usar el bloque try/catch clásico de programación.

JavaScript

```
const obtenerDatos = async () => {
    try {
        const respuesta = await funcionQuePuedeFallar();
        console.log(respuesta);
    }
};
```

```
    } catch (error) {  
        console.error("¡Ups! Falló algo:", error);  
    }  
};
```

3.3 Errores Típicos

- Usar await fuera de una función async (en scripts antiguos daba error, aunque ahora en módulos modernos se permite "Top-level await").

3.4 Mini-reto (5 min)

Tienes una función login() que devuelve una promesa. Conviértela a async/await y envuélvela en un try/catch para que si falla, imprima "Usuario incorrecto".

4. FETCH API + HTTP + JSON

HTTP: Es el protocolo de la web. Tú (Cliente) envías una **Petición (Request)** y el Servidor envía una **Respuesta (Response)**.

JSON: El formato de texto estándar para intercambiar datos. Parece un objeto JS, pero es texto plano.

4.1 Métodos HTTP (Verbos)

Son formas de usar las llamadas fetch, en función de nuestra intención se usan unas u otras.

- **GET:** Traer datos (Leer).
- **POST:** Enviar datos nuevos (Crear).
- **PUT/PATCH:** Actualizar datos (Editar).
- **DELETE:** Borrar datos (Eliminar).

4.2 Ejemplos con Fetch

Usaremos <https://jsonplaceholder.typicode.com> (una API fake gratuita).

4.2.1 Petición GET Básica

JavaScript

```
<body>  
  <h1>pedir usuario</h1>  
  <form action="">  
    <input type="text" id="usuario">
```



```

        <input type="button" id="go" value="Go!">
    </form>
    <script >
        //variables
        let id=document.getElementById("usuario")
        let boton=document.getElementById("go")

        //eventos
        boton.addEventListener("click", (e) =>{
            if (id.value>0)
                obtenerUsuario(id.value);
        })

        const obtenerUsuario = async (id) => {
            try {
                // 1. Lanzar la petición (fetch devuelve una promesa)
                const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

                // 2. Verificar si HTTP fue exitoso (status 200-299)
                if (!response.ok) {
                    throw new Error(`Error HTTP: ${response.status}`);
                }

                // 3. Convertir el cuerpo de texto JSON a Objeto JS
                // .json() TAMBIÉN es una promesa (asíncrono)
                const data = await response.json();

                console.log(`Usuario: ${data.name} - Email: ${data.email}`);

            } catch (error) {
                console.error("Fallo de red o servidor:", error.message);
            }
        };

        //obtenerUsuario(9999); // Error 404
        //obtenerUsuario(1); // Usuario 1
    </script>
</body>

```



4.2.2 Petición POST (Crear)

JavaScript

```

const crearPost = async () => {
    const nuevoPost = {

```

```
    title: 'Mi nuevo post',
    body: 'Contenido asombroso',
    userId: 1,
  };

  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/posts', {
      method: 'POST', // Importante cambiar el método
      headers: {
        'Content-Type': 'application/json', // Avisamos que enviamos JSON
      },
      body: JSON.stringify(nuevoPost) // Convertir Objeto JS -> Texto JSON
    });

    const data = await response.json();
    console.log("Post creado con ID:", data.id);

  } catch (error) {
    console.error(error);
  }
};
```

4.3 Errores Típicos

- Olvidar `await response.json()`. Si haces `console.log(response)`, verás el objeto respuesta crudo, no tus datos.
- Creer que `catch` captura errores 404. **Corrección:** `fetch` solo rechaza si **no hay red**. Un error 404 se considera "éxito de red" (el servidor respondió). Debes verificar `response.ok` manualmente.

4.4 Mini-reto (5 min)

Crea una función `borrarPost(id)` que haga un `fetch` con `method: 'DELETE'`.

Si `response.ok` es `true`, imprime "Post borrado correctamente".

5. CHEAT SHEET

- **Esperar:** `setTimeout(() => {}, ms)` (No bloquea).
- **Promesa:** Objeto con estados `Pending` -> `Resolved` / `Rejected`.
- **Consumir:** `.then(data => ...).catch(err => ...)` o `try { await ... } catch (err) { ... }`.
- **Fetch GET:** `fetch(url)`.
- **Fetch POST:** `fetch(url, { method: 'POST', body: JSON.stringify(data) })`.

- **Parsear JSON:** `const data = await response.json();`
- **Error HTTP:** Siempre chequea `if (!response.ok)`.

6. PREGUNTAS RÁPIDAS (V/F)

1. JavaScript ejecuta código síncrono antes que asíncrono. (V/F)
2. `setTimeout(fn, 0)` se ejecuta antes que la siguiente línea de código. (V/F)
3. `await` bloquea toda la página web hasta que termina. (V/F)
4. `fetch` lanza un error en el `.catch` si recibe un 404. (V/F)
5. `JSON.stringify` convierte un objeto JS a texto. (V/F)
6. Una Promesa puede pasar de Rejected a Resolved. (V/F)
7. El método por defecto de `fetch` es GET. (V/F)
8. `async` hace que una función devuelva automáticamente una promesa. (V/F)

7. CHECKLIST DE HABILIDADES

- ☐ Entiendo la diferencia entre Pila (Stack) y Cola (Queue).
- ☐ Sé crear una Promesa y consumirla con `.then` y `await`.
- ☐ Sé hacer peticiones a una API real con `fetch`.
- ☐ Sé manejar errores de red y errores 404/500.
- ☐ Sé enviar datos JSON a un servidor.

8. EJERCICIOS FINALES (LA PRÁCTICA)

Restricciones: Usa `const/let`, `arrow` functions y la API `https://jsonplaceholder.typicode.com`.

8.1 Nivel 1: Calentamiento (Fácil)

1. **El Temporizador:** Crea una función `esperar(ms)` que devuelva una promesa que se resuelva después de `ms` milisegundos. Úsala con `await` para imprimir "Inicio", esperar 2 segundos y luego "Fin".
2. **Promesa Aleatoria:** Crea una promesa que tenga 50% de probabilidad de

resolverse ("Éxito") y 50% de rechazarse ("Error"). Consúmela con `.then/.catch`.

3. **Fetch Simple:** Haz un GET a `/users/1` e imprime solo el nombre y la ciudad del usuario.
4. **JSON Stringify:** Crea un objeto `producto = { id: 1, nombre: "TV" }`. Conviértelo a string JSON e imprímelo. Luego haz lo inverso (`JSON.parse`).
5. **Orden de Logs:** Sin ejecutar, escribe el orden de salida de:

JavaScript

```
console.log(1);  
fetch('...').then(() => console.log(2));  
console.log(3);
```

8.2 Nivel 2: Intermedio (Flujos Reales)

6. Encadenando Promesas:

- Haz fetch al `/users/1`.
- Toma su id y haz OTRO fetch a `/posts?userId=1`.
- Imprime los posts del usuario.
- *Pista:* Usa dos `await` seguidos o encadena dos `.then`.

7. Async/Await con Error:

- Haz una función `getUsuario(id)`.
- Si el id es `> 10`, lanza un error manual `throw new Error("ID inválido")`.
- Usa `try/catch` para manejarlo.

8. Carrera de Servidores:

- Crea 3 promesas con `setTimeout` aleatorios (ej: 500ms, 1000ms, 200ms).
- Usa `Promise.any` para ver cuál gana e imprime "Ganó la promesa X".

9. Carga Paralela (Performance):

- Usa `Promise.all` para traer `/users/1` y `/todos/1` al mismo tiempo.
- Imprime un objeto combinado `{ usuario: ..., tarea: ... }`.

10. Validador de URL:

- Haz una función que reciba una URL.

- Intenta hacer fetch. Si da error de red o 404, devuelve false. Si es ok, devuelve true.

11.El Buscador de Usuario:

- Input HTML y Botón.
- Al clickar, busca el usuario por ID en la API.
- Si existe, pinta su nombre en el DOM. Si no (404), pinta "Usuario no encontrado" en rojo.

12.Poster:

- Haz una función que envíe un POST a /posts.
- Datos: { title: "Test", body: "Prueba", userId: 1 }.
- Muestra en consola la respuesta del servidor (debería devolver el objeto con un id nuevo 101).

8.3 Nivel 3: Boss Level (Mini-Cliente API)

13.Dashboard de Usuarios (AllSettled):

- Tienes una lista de IDs [1, 2, 9999, 3] (el 9999 fallará).
- Usa .map para crear un array de promesas de fetch.
- Usa Promise.allSettled. (busca en internet)
- Filtra y muestra solo los nombres de los usuarios que se cargaron bien. Ignora los errores sin que explote la app.

14.Feed con "Retry" (Lógica de Reintento):

- Crea una función fetchConRetry(url, intentos).
- Intenta hacer fetch. Si falla (catch), espera 1s y vuelve a intentar restando 1 intento.
- Si los intentos llegan a 0, lanza error definitivo.
- *Pista:* Usa recursividad o un bucle while con await esperar(1000).

15.El "Smart Search":

- Input de búsqueda que busca posts por texto (simulado, filtra localmente o busca endpoint si existe).

- **Debounce:** Si escribo "H", "Ho", "Hol", "Hola" rápido... solo debe lanzarse LA ÚLTIMA petición fetch, cancelando o ignorando las anteriores.
- *Pista:* Usa una variable timeoutId global y clearTimeout antes de lanzar el nuevo fetch.

¡A conectar el mundo!

9. ALMACENAR INFORMACIÓN EN EL NAVEGADOR

9.1 Localstorage y Sessionstorage

En el navegador, puedes almacenar información utilizando **Web Storage**, que incluye dos opciones principales: **localStorage** y **sessionStorage**. Ambas te permiten guardar datos en forma de clave-valor directamente en el navegador del usuario, pero tienen diferencias:

- **localStorage:** Almacena datos que permanecen incluso si cierras el navegador y lo vuelves a abrir.
- **sessionStorage:** Almacena datos solo mientras la pestaña del navegador está abierta. Cuando cierras la pestaña, los datos se eliminan.

9.2 Ejemplo sencillo con localStorage

Supongamos que quieres guardar el nombre de un usuario en el navegador.

- Guardar:

```
localStorage.setItem('nombreUsuario', nombre); // Guarda el valor con la clave nombreUsuario
```

- Recuperar:

```
const nombreGuardado = localStorage.getItem('nombreUsuario'); // Recupera el valor
```

- Eliminar:

```
localStorage.removeItem('nombreUsuario');
```

```
<body>
  <h1>Guardar información en el navegador</h1>
  <input type="text" id="nombre" placeholder="Escribe tu nombre">
  <input type="button" id="guardar" value="Guardar Nombre">
  <input type="button" id="mostrar" value="Mostrar Nombre">
  <input type="button" id="borrar" value="Borrar Nombre">
  <p id="resultado"></p>

  <script>
    let nombre = document.getElementById("nombre")
    let guardar = document.getElementById("guardar")
```

```

let mostrar = document.getElementById("mostrar")
let borrar = document.getElementById("borrar")
let resultado = document.getElementById("resultado")

// Botón para guardar el nombre
guardar.addEventListener('click', () => {
  const nombreUsuario = nombre.value // Obtiene el valor del input
  localStorage.setItem('nombreUsuario', nombreUsuario) // Guarda el nombre en
localStorage
  alert('Nombre guardado en localStorage')
})
// Botón para guardar el nombre
borrar.addEventListener('click', () => {
  localStorage.removeItem('nombreUsuario') // Guarda el nombre en
localStorage
  resultado.textContent = ""
  alert('Nombre borrado en localStorage')
})

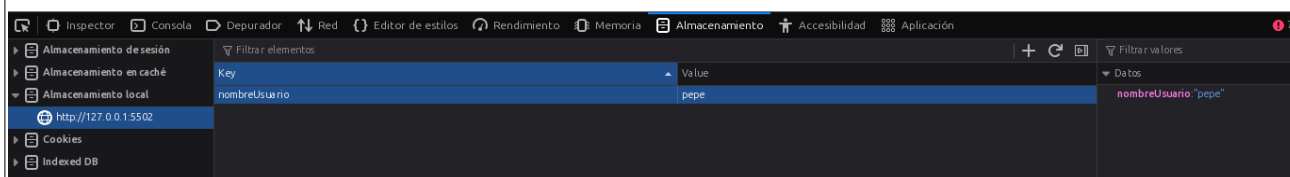
// Botón para mostrar el nombre guardado
mostrar.addEventListener('click', () => {
  const nombreGuardado = localStorage.getItem('nombreUsuario') // Recupera el
nombre de localStorage
  if (nombreGuardado) {
    resultado.textContent = `Nombre guardado: ${nombreGuardado}`
  } else {
    resultado.textContent = 'No hay ningún nombre guardado.'
  }
})
</script>
</body>

```

Guardar información en el navegador

pepe

Nombre guardado: pepe



9.3 Ejemplo sencillo con sessionStorage

El funcionamiento es similar al de localStorage, pero los datos se **eliminan al cerrar la pestaña del navegador**. Solo necesitas reemplazar localStorage por sessionStorage en el código anterior.

```

sessionStorage.setItem('clave', 'valor'); // Guardar información
const dato = sessionStorage.getItem('clave'); // Recuperar información

```

```
sessionStorage.removeItem('clave'); // Eliminar información específica  
sessionStorage.clear(); // Eliminar todo
```

10. PRÁCTICAS INTEGRADORAS.

Los siguientes ejercicios están planteados por una IA, combinan los tres módulos vistos, Lógica (Javascript 1), DOM (Javascript 2) y Asincronía/APIs (Javascript 3). Son para ampliar en el caso de que necesites retos!.

La clave para que estas prácticas sean "Nivel Pro" es seguir la **Regla de Oro**:

1. **Trae los datos** (Async/Fetch).
2. **Guárdalos en memoria** (Array/Estado).
3. **Pinta el HTML** basándote en esos datos (DOM).
4. **Si filtras/modificas**, lo haces en el Array y vuelves a pintar.

10.1 El Buscador de Usuarios "En Vivo" (RandomUser API)

Objetivo: Consumir una API real, transformar datos y filtrar en el cliente.

- **API:** <https://randomuser.me/api/?results=50>
- **Lógica (JS1+J22):**
 - Haz fetch. Usa async/await.
 - Guarda los usuarios en un array global let usuarios = [].
 - Mapea los datos para quedarte solo con lo que te sirve: { nombre, foto, ciudad }.
- **DOM (M4):**
 - Píntalos en una lista.
 - Escucha el evento input de un buscador.
 - Filtra el array usuarios.filter() por nombre.
 - **Truco Pro:** Borra la lista (innerHTML = "") y repinta los filtrados en cada letra.

10.2 La Pokedex (Promise)

- **API:** <https://pokeapi.co/api/v2/pokemon?limit=20>

- **Lógica :**
 - Primero haz un fetch para obtener la lista de los 20 (solo te da nombre y URL).
 - **El Reto:** Tienes que hacer *otra* petición por cada Pokémon para sacar su imagen.
 - Usa .map() para crear un array de promesas (fetch(url)).
 - Usa await Promise.all(arrayDePromesas) para esperarlas todas a la vez.
- **DOM :**
 - Pinta 20 tarjetas *de golpe* solo cuando todas las imágenes estén listas.

10.3 Dashboard Crypto con "Auto-Refresh"

Objetivo: Manejar actualizaciones periódicas y comparación de datos.

- **API:** <https://api.coincap.io/v2/assets?limit=5>
- **Lógica :**
 - Haz una función cargarPrecios().
 - Usa setInterval para que se ejecute cada 5 segundos.
 - Usa Intl.NumberFormat (JS nativo) para formatear dinero bonito.
- **DOM :**
 - Si el precio subió respecto a la última vez, pinta el texto en verde. Si bajó, en rojo.
 - Necesitarás guardar los precios anteriores en una variable para comparar.

10.4 Fake E-commerce: Carrito Persistente

Objetivo: Simular un flujo de compra completo con categorías.

- **API:** <https://fakestoreapi.com/products>
- **Lógica :**
 - Trae los productos.
 - Usa Object.groupBy (ES2024) para separarlos por category.
- **DOM :**

- Crea una sección (div) por cada categoría.
- Botón "Añadir": Agrega el producto a un array carrito.
- Actualiza el "Total: \$XX" usando .reduce() cada vez que se toque el carrito.

10.5 GitHub User Finder (Manejo de Errores 404)

Objetivo: Aprender a gestionar cuando la API dice "No existe".

- **API:** `https://api.github.com/users/{username}`
- **DOM:** Input text + Botón "Buscar".
- **Lógica :**
 - Haz fetch dinámico con el valor del input.
 - **Manejo de Error:**
 - Si `response.status === 404`, lanza un error y pinta una tarjeta roja "Usuario no encontrado".
 - Si `response.ok`, pinta la foto y la bio del usuario.
- **Extra :** Usa Destructuring anidado para sacar datos profundos del JSON.

10.6 Blog con "Optimistic UI" (POST & DELETE)

Objetivo: Simular que la app es rapidísima, actualizando el DOM antes de que el servidor responda.

- **API:** `https://jsonplaceholder.typicode.com/posts`
- **Acción 1 (Borrar):**
 - Al clicar "Borrar", elimina el post del DOM **inmediatamente** (`element.remove()`).
 - LUEGO lanza el `fetch(url, { method: 'DELETE' })`.
 - Si el fetch falla (catch), muestra un `alert("Error")` y vuelve a pintar el post (deshacer).
- **Acción 2 (Crear):**
 - Formulario para crear post (POST).
 - Al enviar, agrégalo visualmente al principio (prepend).

10.7 Conversor de Monedas (Math + Objetos)

Objetivo: Manipular objetos de tasas de cambio.

- **API:** <https://api.frankfurter.app/latest?from=USD> (Pública).
- **Lógica:**
 - Recibes un objeto rates: { EUR: 0.93, GBP: 0.79... }.
 - Guarda esas tasas en una variable.
- **DOM:**
 - Dos inputs: Cantidad USD y Select de Moneda destino.
 - Evento input: Calcula en tiempo real cantidad * tasa[moneda].

10.8 El "Smart Search" con Debounce (Nivel Boss)

Objetivo: Evitar saturar la API con peticiones innecesarias.

- **Escenario:** Buscador de películas (<http://www.omdbapi.com/> o simulado).
- **Problema:** Si escribo "Batman", no quiero hacer 6 peticiones ("B", "Ba", "Bat"...).
- **Lógica :**
 - Crea una variable global let timeoutId.
 - En el evento input, lo primero que haces es clearTimeout(timeoutId).
 - Luego, timeoutId = setTimeout(() => { buscarPeli(...) }, 500).
 - **Resultado:** Solo busca cuando el usuario deja de escribir por 0.5s.

10.9 La Carrera de Servidores (Promise.any)

Objetivo: Entender la redundancia de servidores.

- **Simulación:** Crea 3 funciones que retornen promesas con setTimeout aleatorios. Llámalas Servidor A, Servidor B, Servidor C.
 - Haz que una de ellas siempre falle (reject).
- **Lógica:**
 - Usa Promise.any([p1, p2, p3]).
- **DOM:**
 - Muestra un "Cargando..."

- En cuanto responda la primera (Any), oculta el loading y muestra: "Datos cargados desde: [Nombre Servidor]".

10.10 Gestor de Tareas con Estados (Async + ClassList)

Objetivo: Sincronizar estado visual con estado lógico asíncrono.

- **API:** https://jsonplaceholder.typicode.com/todos?_limit=10
- **Lógica:**
 - Trae las tareas. Cada una tiene completed: true/false.
- **DOM:**
 - Píntalas. Si completed es true, añade la clase CSS .tachado.
 - Al hacer click en una tarea:
 1. Cambia visualmente la clase (toggle).
 2. Lanza un fetch (PATCH) para actualizar el estado en el "servidor".
 3. Si falla, revierte el cambio visual.

¡Elige una y empieza a picar código!

11. BIBLIOGRAFÍA

Aquí tienes los enlaces originales correspondientes a esos archivos, organizados por autor, para que puedas consultar la fuente directa:

1. Documentación Oficial y Guías (Base Teórica)

Gran parte de la estructura técnica y las definiciones precisas (especialmente del Módulo 1 y 6) provienen de la documentación de Google Developers.

- **Web.dev - Learn JavaScript:** <https://web.dev/learn/javascript/>

2. Midudev (JavaScript Moderno y ES2023/24)

La parte de "Modern Syntax", novedades como Object.groupBy, toSorted, async/await y buenas prácticas actuales se basan en su contenido.

- **Canal de YouTube:** [midudev](https://www.youtube.com/c/midudev)
- **Cursos/Web:** midu.dev

3. MoureDev (Fundamentos Sólidos)

Las explicaciones sobre tipos de datos, estructuras y la lógica inicial para principiantes se apoyan en su curso.

- **Curso JavaScript desde Cero:** [Ver en YouTube](https://www.youtube.com/watch?v=...)
- **Web:** mouredev.com

4. SoyDalto (Nivel Junior y DOM)

Muchos de los conceptos sobre el DOM (Módulo 4), historia de JS y flujo de ejecución provienen de su curso nivel Junior.

- **Curso de JavaScript (Nivel Junior):** [Ver en YouTube](https://www.youtube.com/watch?v=...)

5. Sergie Code (Enfoque Práctico)

Los ejemplos didácticos y el enfoque paso a paso para principiantes absolutos.

- **Curso Gratis JavaScript:** [Ver en YouTube](https://www.youtube.com/watch?v=...)